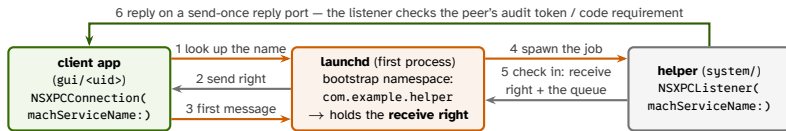


In one line: **XNU** is a *hybrid* kernel. CMU's **Mach** (tasks, threads, **ports**, VM, scheduler), **BSD** code from FreeBSD (processes, POSIX, VFS, sockets, signals) and **IOKit** (C++ drivers) in *one* address space. The kernel starts **launchd** as the first process; launchd owns every service *name*, holds its port, and **starts the job when the first message arrives**. **XPC** is launchd-managed IPC on top — how an app splits into small, separately sandboxed, crash-isolated helpers.

On-demand launch: client → launchd → daemon, over Mach ports



Idle → the job exits (an XPC service gets SIGKILL), the service port stays live, the next message relaunches it · crash → the client's interruptionHandler; state is lost · requests wait for the launch instead of failing · launchd exiting = kernel panic "initproc exited"

mach_msg copies the header + inline bytes, *moves* or *makes* port rights, virtual-copies out-of-line memory, appends a trailer

User space down to XNU (source dir)

apps · agents · daemons · XPC services · DriverKit dexts		
Foundation · libdispatch · libSystem · libc, libxpc		
BSD syscalls	Mach traps (mach_msg = 31)	
BSD bsd/ proc, VFS, sockets, signals, memorystatus	Mach osfmk/ task, thread, port, VM, scheduler	IOKit iokit/ libkern/ C++, IORegistry
security/ MAC policy hooks · pexpert/ platform · libsa/ boot		

all of it one kernel, one address space

Mach vs BSD — one process, two views

	Mach (<i>osfmk/</i>)	BSD (<i>bsd/</i>)
unit runs talks names faults	task: VM map + port space thread ports + mach_msg port name, local to one task exception → thread, then task, then host exception port	proc: pid, credentials, fds → 1:1 with its task a pthread is a Mach thread pipes, sockets, signals pid — reused ; add token adds a <i>version</i> signal: BAD_ACCESS → SEGV (bad address) or BUS · BAD_INSTRUCTION → ILL · ARITHMETIC → FPE · BREAKPOINT → TRAP

Ports, rights, messages

A **port** is a kernel message queue; tasks hold *rights* to it under task-local names: **receive** (one holder, who dequeues) · **send** (many) · **send-once** (one message — replies) · **port set** (wait on many, like `select`) · **dead name**.

header bits · size remote = destination local = reply port voucher · id	descriptors (if COMPLEX) port · OOL memory OOO ports · guarded	body inline bytes (copied)	trailer (kernel) audit token: audit, euid, egid, ruid, rgid, pid, asid, pidversion
--	--	---	--

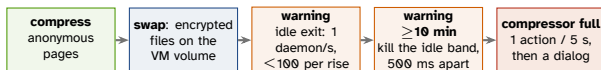
rights travel as MOVE_RECEIVE · MOVE_SEND · MOVE_SEND_ONCE · COPY_SEND · MAKE_SEND · MAKE_SEND_ONCE; OOL memory as a *virtual* (copy-on-write) or *physical* copy

- **Task ports**, capability strictly decreasing: control → read → inspect → name; also `debugstrap` (launched), `host`, `exception`. Control ports are **pinned**. `task_for_pid`: debugger entitlement (admin dialog, 10-hour session) + a target with `get-task-allow`; SIP-protected processes refuse.
- **Hardened** (platform binaries): service ports' receive right is immutable; a reply port's `send-once` right lets exactly one reply back, from that peer. Breach = guard exception.
- **MIG** generates RPC stubs; XNU's docs discourage new raw Mach code — use XPC.

IOKit, kexts, DriverKit

Devices and drivers sit in the **IOWireless** (iowireless). "Kexts are no longer recommended": since macOS 11 third-party kexts load only at boot, from an *Auxiliary Kernel Collection* — on Apple silicon only in Reduced Security (TR + admin password). **DriverKit** (macOS 10.15; M-series iPads, iPadOS 16): .dext drivers in user space, shipped inside an app — on the Mac installed as system extensions.

Memory pressure – macOS vs iOS



- **normal · warning · critical** reach code via DispatchSource.makeMemoryPressureSource: relax caching / drop all caches. PROC_LIMIT_WARN fires at 80 % of a process's limit.
- **Idle exit** takes only opted-in (EnablePressuredExit), idle (no open transaction) daemons. **Compressor full**: kill the largest process if it holds > ½ of compressed pages, else a pcontrol action (kill, suspend, throttle); then "Out of Application Memory".
- Past a *hard* footprint limit: killed at once; past a *soft* one: a simulated crash.
- **Jetsam** is the same subsystem everywhere: 210 priority levels, every process but launchd has one (0 idle - 100 foreground); RunningBoard moves apps between bands; most kills climb from the bottom. iOS swap is the **freezer** (suspended apps' dirty memory to flash, band 75), plus **app swap** on M1-and-later iPads.

Traps

- ! "XNU is a microkernel" — Mach, BSD and IOKit run in one kernel address space.
- ! Daemonizing (daemon(3), fork + parent exits): launchd sees the job die.
- ! KeepAlive true + a crashing binary = a respawn every 10 s.
- ! setuid() in a root daemon is not "being the user": bootstrap subset and audit session state — use an agent.

launchd — first process (OS X 10.4)

The kernel execs /sbin/launchd. **Daemon:** one instance for all clients, system/, root, no UI. **Agent:** one per logged-in user, gui/<uid> (= login/<asid>), may show UI. pid/<pid> = the XPC services that process sees. <Label>.plist lives in ~/Library/LaunchAgents / Library/LaunchAgents/Daemons (admin) / System/Library/... (Apple).

Label	required; the unique job name
ProgramArguments	argv for execvp — no shell
BundleProgram	path inside the app bundle (SMAppService only)
StartOnLoad	Start once when loaded
KeepAlive	true, or SuccessfulExit - Crashed - PathState... ORed
MachineServices	bootstrap names; the job must check in
Sockets	launchd listens; launch_activate_socket hands over fds
StartInterval	every N s — a firing during sleep is missed
StartCalendarInterval	cron-like — missed in sleep → once at wake
ThrottleInterval	spawn at most every N s (default 10)
ProcessType	Background - Standard - Adaptive - Interactive
EnablePresuredExit	reclaimable when idle under pressure; ignored with KeepAlive true

Stop: SIGTERM, SIGKILL after `ExitTimeOut` — at once if no transaction is open (a message expecting a reply holds one). **macOS 27:** launchd refuses quarantined plists.

XPC – three kinds of service

	launch agent	launch daemon	XPC service (bundled)
instances runs as	1 per logged-in user that user	1 system-wide root	1 per client, dies with it the app's user; sandboxed by default; never root
lives in reach	LaunchAgents plist machServiceName:	LaunchDaemons plist +.privileged	Contents/XPCServices/X.xpc serviceName: — that app only

- A daemon **cannot open** a connection to a user process — it only answers. `launchd` SIGKILLS an idle XPC service; keep it stateless.
- **APTs:** `C libxpc (10.7)`, `NSXPCConnection (10.8)`; `asobjc` protocol, void methods, one reply block, `NSSecureCoding` arguments - `Swift XPCSession / XPCListener` (macOS 14): Codable. The wire encoding is no ABI — never persist it.
- **Interruption:** the peer exited or crashed — the connection still works, the next message relaunches it, state is gone. **Invalidation:** never connected, or invalidated — always the last handler; build a new connection.
- **Check the peer**, not its pid — pids are reused and pid-then-lookup races; the audit token carries the pid version. Code requirement: `C API (macOS 12)`, `setCodeSigningRequirement (13)`, lightweight requirement (14.4), `XPCPeerRequirement.isFromSameTeam().hasEntitlement(:) (26)`.

```

struct Job: Codable { let png: Data; let px: Int }
struct Thumb: Codable { let data: Data }

// bundled service, main.swift
let b = try XPCLListener(service: "com.example.app.Resizer") { req in
    req.accept { (job: Job) in Thumb(data: render(job)) } }
dispatchMain()

// app: the first message launches the service
let s = try XPCService(svc: "com.example.app.Resizer")
let t: Thumb = try s.sendSync(Job(png: png, px: 512))

```

SMAAppService — shipping a helper

macOS 13+; replaces SMJobBless (deprecated in 13) and SMLogItemSetEnabled. The plist stays in the bundle — Contents/Library/LaunchDaemons/ (or LaunchAgents) with BundleProgram — and SMAppService.daemons(plistName: "com.example.helper.plist").register(). status: notRegistered · enabled · requiresApproval · notFound. A daemon needs the user's authentication; approving it approves the bundle's other helpers. One switch per app in Login Items; a plist outside the bundle joins its app's switch with AssociatedBundleIdentifiers; sftool dumptbpm lists them all. macOS 26+: tasks still running after the app quits trigger an allow / don't allow prompt.

Remember

Mach moves messages, BSD owns processes, launchd owns names — helpers start when called, die when idle, and trust audit tokens, not pids.

Related: [os-fundamentals](#) · [ios-sandbox-security-model](#) (entitlements, sandbox) · [mach-o-and-dyld](#) · [macos-power-user-cli](#) (launchctl, log) · [macos-security-architecture](#) (SIP, TCC) · [systemd-deep](#) (Linux's launchd) · [linux-memory-oom](#) · [Logging, signposts & power](#) · [Code signing internals](#) · [Crashes & symbolication](#) · [App extensions](#) — [own process](#), [shared disk](#)